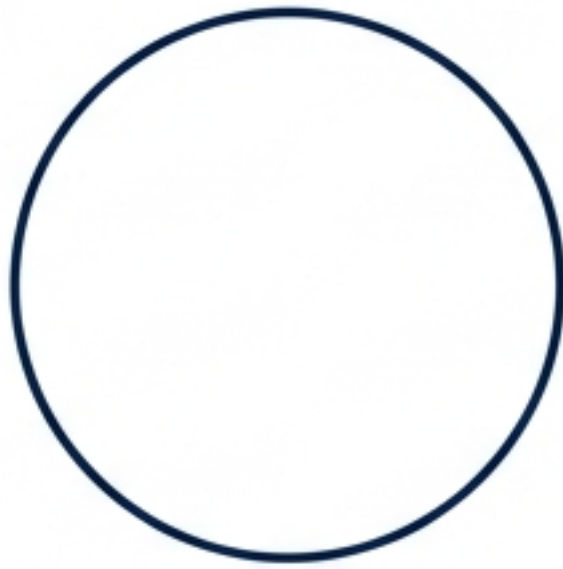


Requirements Specification.

Translating user needs into system blueprints.

The core challenge of specification.



The Goal

Clear, unambiguous, complete,
and perfectly consistent
requirements.



The Reality

Stakeholder conflicts exist.
Interpretations vary.
Ambiguity is inherent.

Specification is the ongoing process of minimizing human interpretation.

Two distinct requirement lenses.

User Requirements

End-users and managers (no technical background assumed).

External system behavior and provided services only.

Natural language, simple tables, no software jargon.

System Requirements

Software engineers and system architects.

Expanded details, functionality, and operational constraints.

Structured specifications, detailed models, expanded technical detail.

Four methods for writing requirements

Natural Language

Numbered sentences. Highly expressive and universal, but potentially ambiguous.

Structured Natural Language

Standard forms and templates. Imposes uniformity and reduces omissions.

Graphical Notations

UML diagrams supplemented by text. Visualizes interactions and component relationships.

Mathematical Specifications

Unambiguous finite-state machines or sets. Extremely precise, but rarely understood by customers.

Rules for natural language specification

Standardize

Invent a uniform format.
Strict rule: One requirement per sentence.

Clarify

Never assume technical knowledge. Remove words like architecture or module. Avoid acronyms.

Justify

Provide rationale. Document the requirement's source to track future changes.

The language of priority.

SHALL

Denotes a mandatory requirement.
The system must support this.

SHOULD

Denotes a desirable requirement.
Not strictly essential.

Moving beyond free-form text.

Standard templates reduce variability and force completeness.



Managing complex alternatives

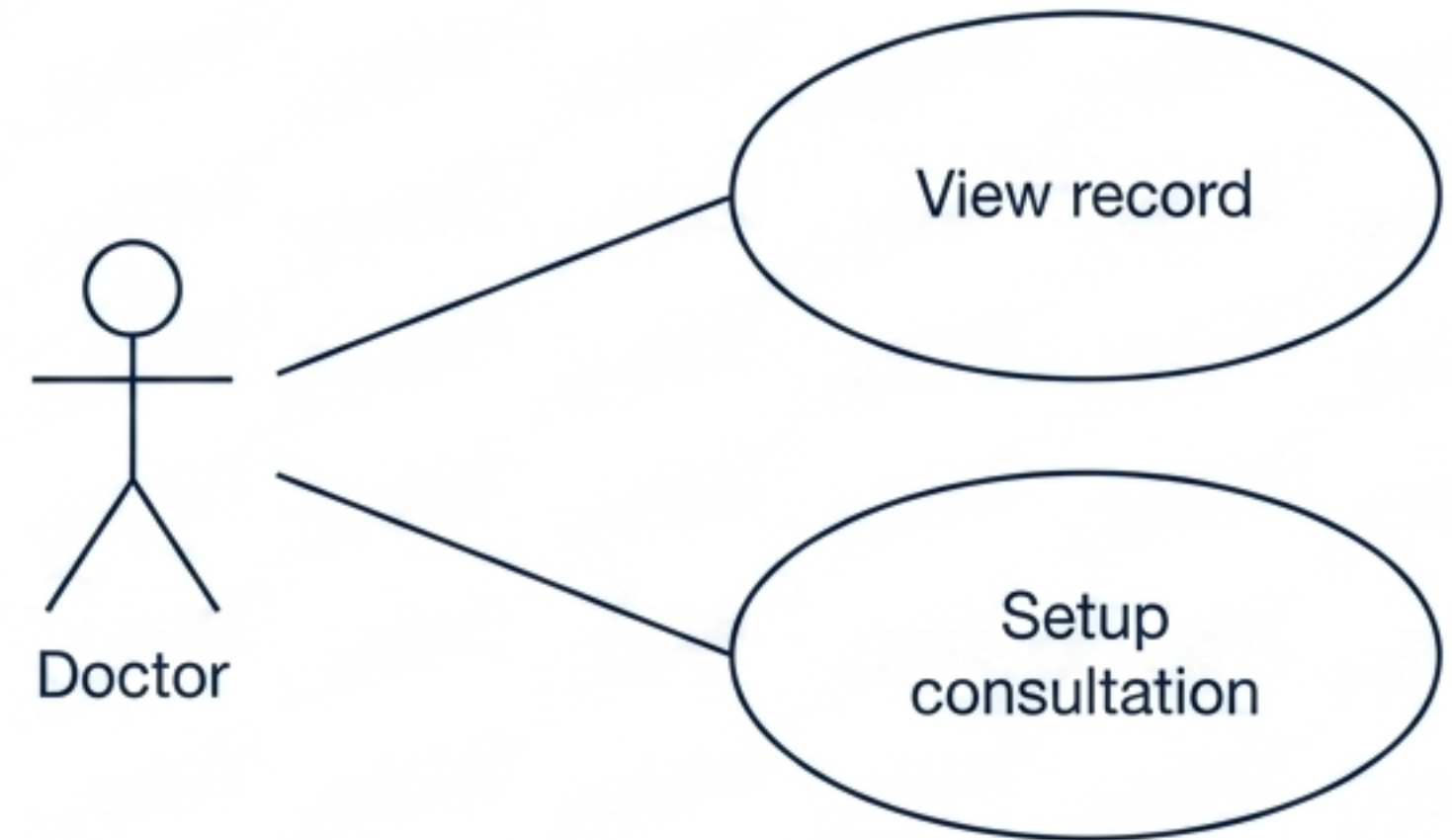
Natural language fails when describing complex, multi-variable logic.

Tables explicitly map shifting Conditions to specific Actions, removing ambiguity.

Condition	→	Action
Sugar level falling	→	CompDose = 0
Sugar level stable	→	CompDose = 0
Sugar level increasing	→	Compute Rate

Modeling explicit user interactions.

Identifying actors and their system privileges.



Most useful for low-level system design. Often too granular for high-level business stakeholders.

The official statement of intent.



The Software Requirements Specification (SRS) dictates exactly what developers must implement.

It serves as the definitive contract for outsourced development and critical systems.

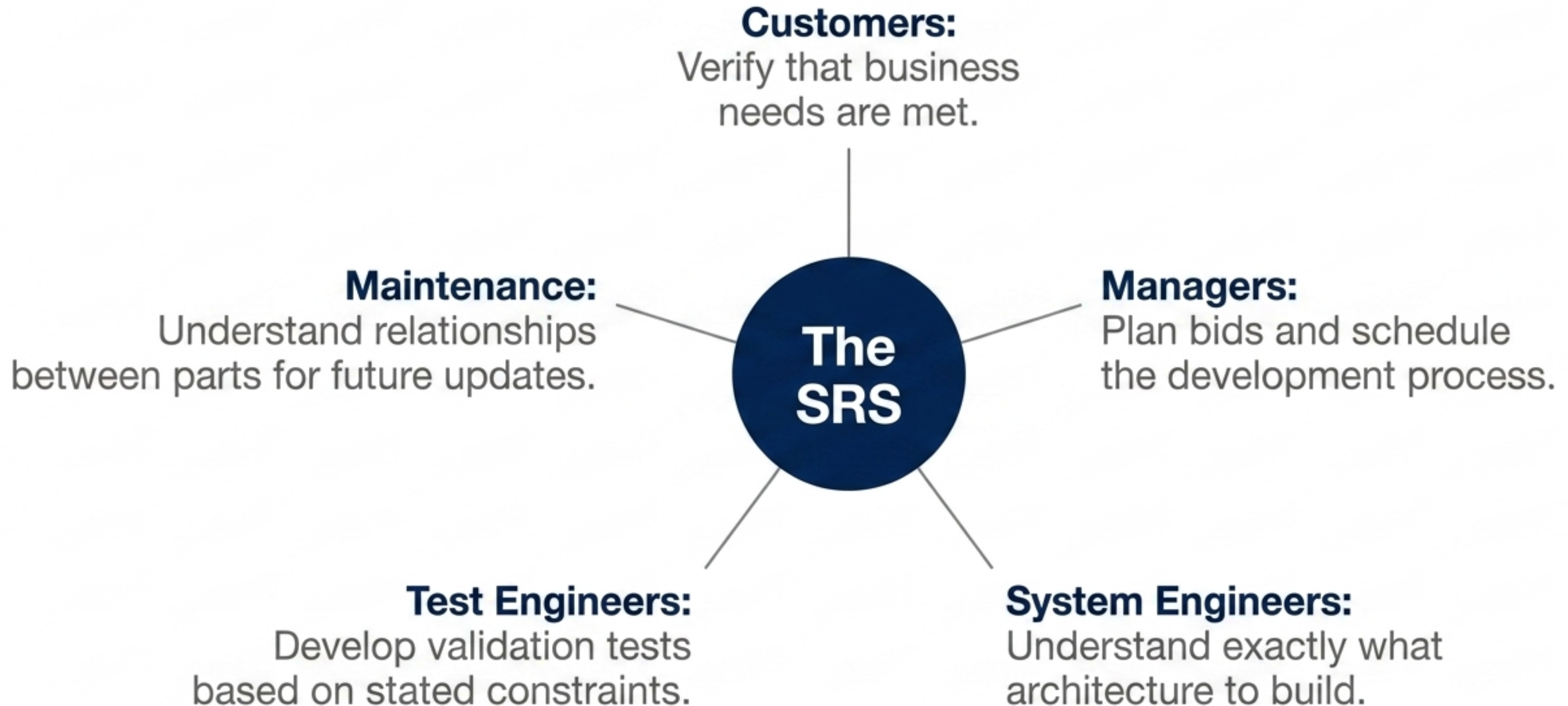
Adapting to the methodology

Stable & Critical Systems: Requires comprehensive, formal, upfront SRS documents to ensure safety and alignment.

Rapid Change: Requirements are collected incrementally on cards as short user stories.

Even in Agile, a short supporting document for overall business and dependability requirements remains essential.

One document, diverse users.



Structuring the specification (IEEE Standard).

Context

(Preface, Introduction, Glossary)

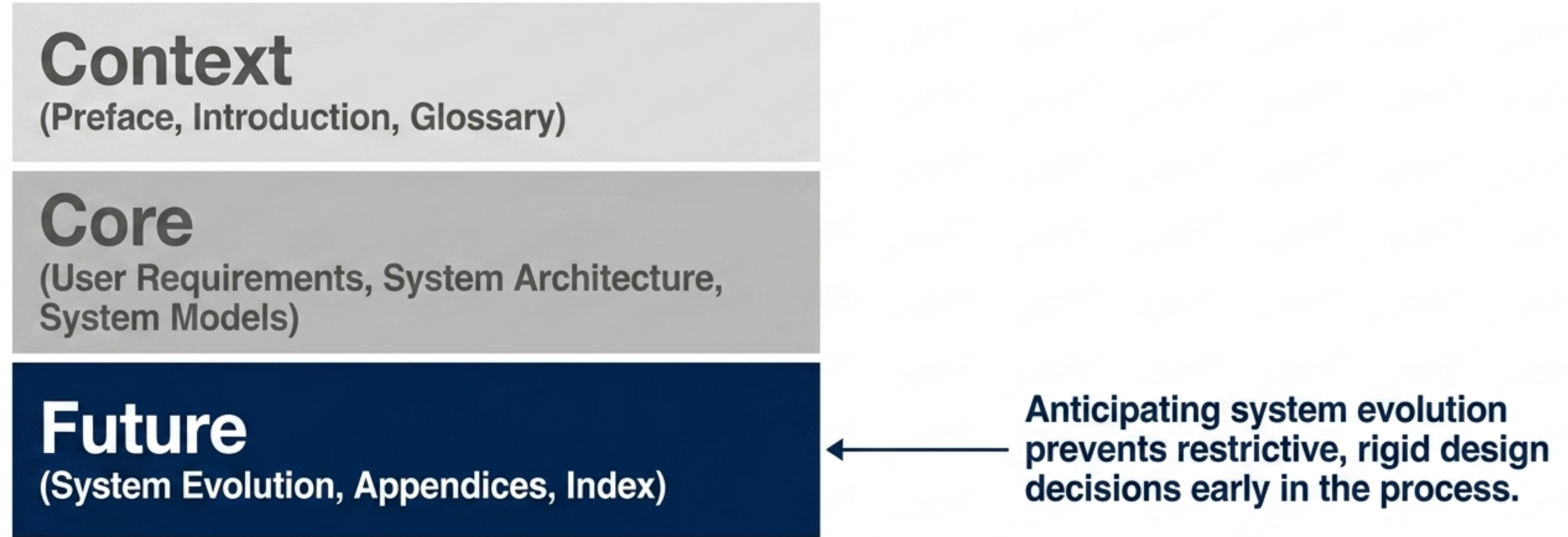
Core

(User Requirements, System Architecture,
System Models)

Future

(System Evolution, Appendices, Index)

Anticipating system evolution
prevents restrictive, rigid design
decisions early in the process.



```
graph TD; Context[Context] --- Core[Core] --- Future[Future]; Future --> Note[Anticipating system evolution prevents restrictive, rigid design decisions early in the process.];
```

The hallmarks of great specification.

- ✓ **Audience-Aware.** Written in the explicit language of the reader (natural vs. structured).
- ✓ **Standardized.** Predictable formats reduce cognitive load and prevent missing variables.
- ✓ **Justified.** Every requirement includes its underlying rationale and traceable source.